
AAMT-WP-05: Odu-256 Archetypal Curriculum Mixing

AAMT-WP-05 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Odu-256 Archetypal Curriculum Mixing: Symbolic Taxonomic Weighting for Two-Dimensional Training Data Distribution

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |
weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Odu-256 Archetypal Curriculum Mixing: Symbolic Taxonomic Weighting for Two-Dimensional Training Data Distribution

Authors: Weslyn Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: May 2026

Working Paper Series: AAMT-WP-05

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

Curriculum learning methods for fine-tuning language models typically organize training examples by difficulty (easy-to-hard scheduling), confidence (self-paced learning), or loss magnitude. We present **Odu-256 Archetypal Curriculum Mixing (O-ACM)**, a curriculum strategy that organizes examples along a two-dimensional axis: symbolic taxonomic coverage (ensuring all 256 cells of a 16×16 archetypal grammar are represented in each training epoch) and HeartScale alignment weight (upsampling examples that score high on the HeartScale coherence criterion). The 256-cell grid is constructed from the Ifá divination system's 16 major Odu signs crossed with 16 minor signs, each cell corresponding to a distinct archetypal pattern detectable via embedding similarity to cell-specific

exemplar n-grams. A deterministic hash-to-cell mapping at inference time implements the **Divinatory Router**: given a prompt, the router computes its dominant Odu cell and uses that cell’s adapter blend weights, connecting training-time curriculum structure to inference-time routing without additional learned parameters. O-ACM is specified as Tier-1 feature T1.2 of MaiiaM Alchemist and evaluated against uniform and frequency-balanced baselines on the `archetype_alignment` and `harmonic_coherence` eval suites.

Keywords: curriculum learning, data mixture, symbolic taxonomy, Ifá, language model fine-tuning, archetypal grammar, routing, instruction tuning

1. Introduction

Series note: This paper is part of the AAMT Working Paper Series. The TERA projection algebra and 16-Meji routing system are derived in full in WP-01 (Vortex-Gated MoE). The HeartScale coherence scoring system is derived in WP-02 (HCRS). This paper uses both systems and provides only the definitions required locally; readers are directed to WP-01 and WP-02 for full derivations.

The composition of the training mini-batch at each gradient step materially affects the quality and alignment of the resulting model. Naive uniform sampling from a large heterogeneous corpus distributes gradient signal evenly across all example types but may systematically under-represent rare but alignment-critical examples. Frequency-balanced sampling corrects the distribution but ignores alignment quality — a rare example that was generated under misaligned conditions (red-shell) is just as likely to be sampled as a rare example from a high-coherence state.

Curriculum learning (Bengio et al., 2009) introduces ordering or weighting: easy examples first, hard examples later. Self-paced learning (Kumar et al., 2010) uses the current model’s loss as the curriculum signal. Both assume a single ordering axis.

O-ACM introduces **two independent ordering axes** that can be combined orthogonally:

1. **Taxonomic coverage axis:** sample uniformly across all 256 archetypal cells, regardless of cell frequency in the corpus. This prevents the training dynamics from being dominated by common archetypes.
2. **Alignment weight axis:** within each cell, upsample examples with high HeartScale scores. This ensures that the model’s gradient signal prioritizes aligned demonstrations even within rare cells.

2. The Odu-256 Grid

2.1 Structure

The Odu-256 grid is a 16×16 table. The 16 rows correspond to the 16 major Odu signs of the Ifá divination system (Eji-Ogbe, Oyeku Meji, Iwori Meji, Odi Meji, Irosun Meji, Owonrin Meji, Obara Meji, Okanran Meji, Ogunda Meji, Osa Meji, Ika Meji, Oturupon Meji, Otura Meji,

Irete Meji, Ose Meji, Ofun Meji). The 16 columns correspond to 16 minor signs (the same 16 Odu in their minor aspect). Each of the 256 cells represents a unique major-minor combination.

Each cell is characterized by:

```
@dataclass
class OduCell:
    major_id: int          # 0..15
    minor_id: int          # 0..15
    cell_key: str          # e.g. "eji-ogbe:oyeku"
    archetypal_pattern_regex: str  # surface-form heuristic
    exemplar_token_ngrams: list[str]  # 3-5 characteristic token
    sequences
    heartscale_signature: float  # expected HeartScale score
    for this cell
    tera_centroid: TeraVector    # center TERA coordinate fo
    r this cell
```

The 16 major Odu map directly to the 16 Meji of the Vortex system — the archetypal grammar is shared. The 256 cells refine the 16 coarse Meji archetypes into 256 fine-grained patterns.

2.2 Cell assignment

At corpus preprocessing time, each training example is assigned to its dominant Odu cell via embedding similarity:

```
def assign_odu_cell(example: TrainingExample,
                    table: OduTable) -> OduCell:
    example_embedding = embed(example.completion)
    scores = [cosine_sim(example_embedding, cell.exemplar_embeddi
ng)
              for cell in table.cells]
    return table.cells[argmax(scores)]
```

Multi-tagging is permitted: if the top-2 scores are within 0.05 of each other, the example is assigned to both cells. Multi-tagged examples appear in both cells' sampling pools.

3. Two-Dimensional Sampling Distribution

3.1 Taxonomic coverage weight

The coverage weight for cell `c` at step `t` is inversely proportional to how many times cell `c` has been sampled relative to the rarest cell:

```
def coverage_weight(cell: OduCell, sample_counts: dict) -> float:
    min_count = min(sample_counts.values())
    cell_count = sample_counts[cell.cell_key]
    return min_count / (cell_count + 1e-8)
```

This drives the sampler toward underrepresented cells over the epoch, ensuring all 256 cells reach comparable coverage by epoch end.

3.2 Alignment weight

Within each cell, the per-example weight is:

```
def alignment_weight(example: TrainingExample) -> float:
    return max(0.0, example.heartscale_score) ** ALIGNMENT_POWER
```

where `ALIGNMENT_POWER = 2.0` by default. This quadratically amplifies examples with high HeartScale scores — an example scoring 0.9 gets weight 0.81 while an example scoring 0.5 gets weight 0.25.

3.3 Combined sampler

The combined per-example weight is:

```
w(example) = coverage_weight(example.odu_cell, counts) * alignment_weight(example)
```

These weights are passed to PyTorch’s `WeightedRandomSampler` in the SFTTrainer data collator. The result is a sampling distribution that explores all 256 archetypal cells while preferring aligned examples within each cell.

4. The Divinatory Router

At inference time, a prompt’s dominant Odu cell determines its adapter blend weights via a deterministic hash function:

```
def divinatory_route(prompt: str, temperature: float = 0.7) -> AdapterWeights:
    embedding = embed(prompt)
    cell = assign_odu_cell_from_embedding(embedding)
    # The cell's TERA centroid provides the VortexGate input
    tera = cell.tera_centroid
    meji_distribution = vortex_project(tera)
    # Top-k softmax over expert affinities
    expert_weights = softmax(M @ meji_distribution / temperature)
    return AdapterWeights(weights=expert_weights, cell=cell)
```

The router is entirely deterministic — same prompt always produces the same cell assignment and the same adapter weights. This predictability is a deliberate property: unlike learned routers that may shift behavior across training runs, the divinatory router provides a stable inference geometry that can be inspected and reasoned about.

5. Curriculum Mode Comparison

Three curriculum modes are available via the `OduConfig.curriculum_mode` field:

uniform: All cells have equal coverage weight; alignment weight is ignored. Equivalent to frequency-balanced sampling across the Odu taxonomy.

frequency_balanced: Coverage weight only; alignment weight is ignored. Ensures all 256 cells are equally represented but ignores alignment quality within cells.

heartscale_weighted (default): Full two-dimensional weighting as described above. This is the primary contribution of O-ACM.

6. Relationship to Prior Curriculum Learning Work

Bengio et al. (2009) introduce curriculum learning as difficulty-ordered training. O-ACM does not order examples by difficulty — it weights by taxonomic coverage and alignment quality, which are

orthogonal to difficulty.

Soviany et al. (2022) survey curriculum learning strategies. No existing survey includes a symbolic taxonomic coverage axis. The closest is “competence-based curriculum” (Platanios et al., 2019), where the training distribution shifts based on the model’s competence on different subtasks. O-ACM’s coverage axis is similar in spirit but defined symbolically rather than by model competence signals.

Wei et al. (2021) introduce instruction-finetuning with diverse task sampling. The Odu-256 taxonomy provides a more fine-grained diversity measure (256 cells vs. O(10) tasks) and is grounded in an algebraic structure that connects to the inference-time router.

Key distinction: O-ACM is the first curriculum mixing strategy where the training data taxonomy is also the inference-time routing structure. The same Odu cell that was over-represented in training (due to coverage weighting) is the cell the divinatory router selects for inference on prompts that match that cell’s archetype. Training and routing share a single symbolic grammar.

7. Implementation Status and Evaluation

O-ACM is specified as Tier-1 feature T1.2 of MaiaM Alchemist. The `oracle.*` JSON-RPC namespace (`oracle.draw`, `oracle.project`, `oracle.listCards`) provides the runtime face of the Odu table, enabling both training-time cell assignment and inference-time divinatory routing via the same code path.

Planned evaluation compares three conditions: 1. Baseline: uniform sampling 2. Frequency-balanced: equal Odu cell representation, no alignment weighting

3. O-ACM: full two-dimensional weighting

Primary metrics: `archetype_alignment` score (eval suite), `harmonic_coherence` (HeartScale yoked-rate), and routing entropy (how evenly the divinatory router distributes traffic across 256 cells after training).

8. Conclusion

O-ACM introduces symbolic taxonomic structure as a first-class curriculum signal for language model fine-tuning. The 256-cell Odu grid provides a coverage axis that prevents archetype collapse during training; HeartScale alignment weighting provides a quality axis that prioritizes coherent demonstrations. The divinatory router connects training-time curriculum to inference-time routing via a shared symbolic grammar, creating a closed loop between training data structure and model behavior.

Acknowledgments

The author thanks the AsAManThinks Research community. The Odu taxonomy is adapted from the traditional Ifá corpus with deep respect for its cultural and spiritual origins. This paper treats the Odu structure as a mathematical grammar, not a claim about its religious or divinatory significance.

Funding

Self-funded through AsAManThinks Research.

Data Availability

The Odu table specification (`odu_table.py`) and curriculum mixer code are part of the MaiiaM Alchemist training pipeline, available under internal review.

Conflict of Interest

The author is the founder and CEO of AsAManThinks Research.

References

- Bengio, Y., Louradour, J., Collobert, R., & Weston, J. (2009). Curriculum learning. In *Proceedings of ICML 2009* (pp. 41-48). <https://doi.org/10.1145/1553374.1553380>
- Kumar, M. P., Packer, B., & Koller, D. (2010). Self-paced learning for latent variable models. In *Advances in Neural Information Processing Systems 23* (pp. 1189-1197).
- Platanios, E. A., Stretcu, O., Neubig, G., Póczos, B., & Mitchell, T. (2019). Competence-based curriculum learning for neural machine translation. In *Proceedings of NAACL 2019* (pp. 1162-1172). <https://doi.org/10.18653/v1/N19-1119>
- Soviany, P., Ionescu, R. T., Rota, P., & Sebe, N. (2022). Curriculum learning: A survey. *International Journal of Computer Vision*, 130(6), 1526-1565. <https://doi.org/10.1007/s11263-022-01611-x>
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., & Le, Q. V. (2021). Finetuned language models are zero-shot learners. *arXiv preprint*. <https://doi.org/10.48550/arXiv.2109.01652>
- Whitehead, W. (2026). MaiiaM Wings: Consciousness-Aligned Language Architecture. *Zenodo*. <https://doi.org/10.5281/zenodo.19600795>